

FPT UNIVERSITY

EMBEDDED LINUX SYSTEMS

LAB-02 REPORT

Device Driver & File System Management

Student	Nguyen Dang Quang
Student ID	SE201072
Group	2
Target	ARMv7-A Cortex-A9 / QEMU vexpress-a9
Date	08 October 2026

1. Student Information and Environment

This report presents the implementation and verification of LAB-02 for the Embedded Linux Systems course. The work extends the Linux 5.15 environment prepared in LAB-01 and focuses on a loadable kernel module, a character device, procfs/sysfs interfaces, raw NAND simulation with JFFS2, and BusyBox automatic module loading.

Student	Nguyen Dang Quang (SE201072)
Group	2
Target board	QEMU vexpress-a9
CPU	ARMv7-A Cortex-A9
Kernel	Linux 5.15.0
Cross compiler	arm-linux-gnueabi-gcc 13.3.0
QEMU	8.2.2
Driver	lab2_driver version 1.0
Device node	/dev/lab2, major 240
Driver buffer	1024 bytes

```

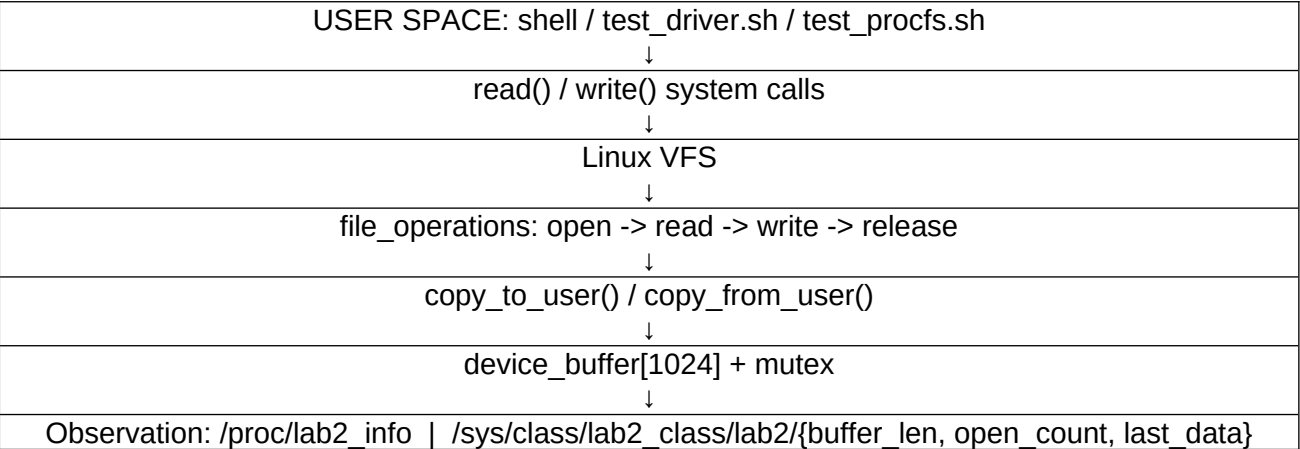
nguyendangquang@nguyendangquang-Bellator:~$ modinfo ./embedded_lab2/driver/lab2_driver.ko | grep -E "filename|version|license|description|vermagic"
filename: /home/nguyendangquang/embedded_lab2/driver/lab2_driver.ko
version: 1.0
description: LAB-02 Character Device Driver
license: GPL
srcversion: 85FA21A222F8302EBBC1CDE
vermagic: 5.15.0 SMP mod_unload modversions ARMv7 gnu
/home/nguyendangquang/embedded_lab2/driver/lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=6347e97d96385ad8a119f857b1aa1e161579e, with debug_info, not stripped
nguyendangquang@nguyendangquang-Bellator:~$

```

Figure 1. Kernel module metadata and ARM EABI5 binary verification.

2. Driver Architecture

The driver provides a character device named /dev/lab2. User applications access the device through normal read() and write() system calls. The Linux VFS dispatches these operations to the file_operations callbacks implemented by lab2_driver. Data is stored in a protected 1024-byte kernel buffer. The same internal state is exposed for observation through /proc/lab2_info and read-only sysfs attributes.



Write data flow: user buffer -> write() -> copy_from_user() -> device_buffer -> procfs/sysfs status.
Read data flow: device_buffer -> copy_to_user() -> user buffer. A mutex serializes access to the shared buffer and related state.

3. Practical Implementation and Results

3.1 Part A - Character Device Driver

The module was cross-compiled for ARM and produced lab2_driver.ko. The module metadata reports version 1.0, GPL license, ARMv7 vermagic for Linux 5.15.0, and an ELF 32-bit ARM EABI5 relocatable object.

```

nguyendangquang@nguyendangquang-Bellator:~$ modinfo -j /embedded_lab2/driver/lab2_driver.ko
file: /embedded_lab2/driver/lab2_driver.ko
filename: /home/nguyendangquang/embedded_lab2/driver/lab2_driver.ko
version: 1.0
description: LAB-02 Character Device Driver
license: GPL
srcversion: 85FA21A22F8302EBBC1CDE
vermagic: 5.15.0 SMP mod_unload modversions ARMv7 p2v8
/home/nguyendangquang/embedded_lab2/driver/lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=e347e697d96385ed9aa19f857b1aa1e0161579e, with debug_info, not stripped
nguyendangquang@nguyendangquang-Bellator:~$

```

Figure 2. lab2_driver.ko module information.

The functional test loads the module, verifies it with lsmod, creates /dev/lab2 using major number 240, writes data, reads the same data back, performs multiple write/read operations, and checks the kernel log. The returned strings match the values written from user space, showing that the read/write callbacks operate correctly.

```

nguyendangquang@nguyendangquang-Bellator:~$ echo "[*] Kernel log"
[*] Kernel log
nguyendangquang@nguyendangquang-Bellator:~$ echo "I create lab2"
I create lab2
nguyendangquang@nguyendangquang-Bellator:~$ echo "=== TEST DONE ==="
=== TEST DONE ===
nguyendangquang@nguyendangquang-Bellator:~$ mknod /dev/lab2 c 240 0
mknod: /dev/lab2: File exists
nguyendangquang@nguyendangquang-Bellator:~$ chmod 777 /dev/lab2
chmod: /dev/lab2: File exists
nguyendangquang@nguyendangquang-Bellator:~$ echo "Hello from userspace, LAB-02!"
Hello from userspace, LAB-02!
nguyendangquang@nguyendangquang-Bellator:~$ write /dev/lab2 "Embedded Linux Lab2 Test Data"
write: /dev/lab2: File exists
nguyendangquang@nguyendangquang-Bellator:~$ cat /dev/lab2
Embedded Linux Lab2 Test Data
nguyendangquang@nguyendangquang-Bellator:~$ lsmod
Module                  Size  Used by
lab2_driver              16384  0
nguyendangquang@nguyendangquang-Bellator:~$ dmesg
[0.000000] lab2_driver: 16384 K - Live 0x7f000000 (0)
[3.123456] Create device node
[3.123456] mknod: /dev/lab2: File exists
[4.123456] Write data
[4.123456] lab2: device opened, open_count=1
[4.123456] lab2: wrote 30 bytes
[4.123456] lab2: device released
[5.123456] Read data
[5.123456] lab2: device opened, open_count=2
[5.123456] lab2: read 30 bytes
[5.123456] Hello from userspace, LAB-02!
[6.123456] lab2: device released
[6.123456] Multiple write/read
[6.123456] lab2: device opened, open_count=3
[6.123456] lab2: wrote 9 bytes
[6.123456] lab2: device released
[6.123456] lab2: device opened, open_count=4
[6.123456] lab2: read 9 bytes
[6.123456] Test 123
[6.123456] lab2: device released
[6.123456] lab2: device opened, open_count=5
[6.123456] lab2: wrote 9 bytes
[6.123456] lab2: device released
[6.123456] lab2: device opened, open_count=6
[6.123456] lab2: read 9 bytes
[6.123456] Test ABC
[6.123456] lab2: device released
[7.123456] [*] Kernel log
[7.123456] lab2_driver: loading out-of-tree module taints kernel.
[7.123456] lab2: registered with major 240
[7.123456] lab2: module loaded successfully
[7.123456] lab2: /proc/lab2_info created
[7.123456] lab2: sysfs attributes created
[7.123456] lab2: module unloaded
[7.123456] lab2: registered with major 240
[7.123456] lab2: module loaded successfully
[7.123456] lab2: /proc/lab2_info created
[7.123456] lab2: sysfs attributes created
[7.123456] lab2: device opened, open_count=1
[7.123456] lab2: wrote 30 bytes
[7.123456] lab2: device released
[7.123456] lab2: device opened, open_count=2
[7.123456] lab2: read 30 bytes
[7.123456] lab2: device released
[7.123456] lab2: device opened, open_count=3
[7.123456] lab2: wrote 9 bytes
[7.123456] lab2: device released
[7.123456] lab2: device opened, open_count=4
[7.123456] lab2: read 9 bytes
[7.123456] lab2: device released
[7.123456] lab2: device opened, open_count=5
[7.123456] lab2: wrote 9 bytes
[7.123456] lab2: device released
[7.123456] lab2: device opened, open_count=6
[7.123456] lab2: read 9 bytes
[7.123456] lab2: device released
[7.123456] === TEST DONE ===
nguyendangquang@nguyendangquang-Bellator:~$

```

Figure 3. Character device functional test and kernel log.

During one recorded test, mknod reported '/dev/lab2: File exists'. This is not a driver failure: rcS had already created the device node during automatic boot initialization. The subsequent chmod, write, read and kernel-log checks completed successfully.

3.2 Part B - procfs and sysfs

The driver creates /proc/lab2_info and three read-only sysfs attributes: buffer_len, open_count and last_data. Before the test write, the proc entry reported Data length = 9, Open count = 6 and Last data = Test ABC. After writing 'Embedded Linux Lab2 Test Data', the values changed to Data length = 30, Open count = 7 and the new Last data. The sysfs attributes reported the same state.

```

nguyendangquang@nguyendangquang-Bellator:~$ cat ~/embedded_lab2/logs/procfs_test.txt
=== LAB-02: procfs & sysfs Test ===

--- /proc/lab2_info BEFORE write ---
Driver name : lab2
Major       : 240
Buffer size : 1024
Data length : 9
Open count  : 6
Last data   : Test ABC

--- Writing data to /dev/lab2 ---
lab2: device opened, open_count=7
lab2: wrote 30 bytes
lab2: device released

--- /proc/lab2_info AFTER write ---
Driver name : lab2
Major       : 240
Buffer size : 1024
Data length : 30
Open count  : 7
Last data   : Embedded Linux Lab2 Test Data

--- sysfs attributes ---
buffer_size : 30
open_count  : 7
last_data   : Embedded Linux Lab2 Test Data

=== procfs/sysfs Test PASSED ===

```

Figure 4. procfs and sysfs verification before and after a write.

3.3 Part C - MTD and JFFS2

Part C was executed on the Ubuntu host using nandsim. The machine already exposed mtd0 as a read-only BIOS NOR device, so it was not modified. The NAND simulator appeared as mtd1. mtdinfo confirmed a 32 MiB NAND device with 16 KiB erase blocks, 512-byte minimum I/O units and writable status.

```

nguyendangquang@nguyendangquang-Bellator:~$ grep -A 16 "===== /proc/mtd =====" ~/embedded_lab2/logs/mtd_jffs2.txt
dev: size erase_size name
mtd0: 02000000 00001000 "BIOS"
mtd1: 02000000 00004000 "NAND simulator partition 0"

===== MTD INFO =====
mtd1
Name:          NAND simulator partition 0
Type:          nand
Eraseblock size: 16384 bytes, 16.0 KiB
Amount of eraseblocks: 2048 (33554432 bytes, 32.0 MiB)
Minimum input/output unit size: 512 bytes
Sub-page size: 256 bytes
OOB size: 16 bytes
Character device major/minor: 90:2
Bad blocks are allowed: true
Device is writable: true

```

Figure 5. NAND simulator and MTD geometry.

The simulated NAND was erased, the JFFS2 image was written with nandwrite, and /dev/mtdblock1 was mounted as JFFS2. The test file contained 'Hello from LAB2 JFFS2'. A second file was created, the filesystem was unmounted and mounted again, and the persistence file remained readable. This demonstrates successful JFFS2 persistence on the simulated raw NAND.

```

nguyendangquang@nguyendangquang-Bellator:~$ grep -A 35 "===== ERASE NAND SIMULATOR =====" ~/embedded_lab2/logs/mtd_jffs2.txt
===== ERASE NAND SIMULATOR =====
Erasing 32768 Kibyte @ 0 -- 100 % complete

===== WRITE JFFS2 IMAGE =====
Writing data to block 0 at offset 0x0

===== LOAD MODULES =====

===== CREATE MOUNT POINT =====

===== MOUNT JFFS2 =====

===== FILES =====
total 5
drwxr-xr-x 3 root    root      0 Jan 1 1970 .
drwxr-xr-x 3 root    root      0 Jan 1 1970 ..
-rw-r--r-- 1 nguyendangquang nguyendangquang 22 Sep 29 19:10 test.txt

===== READ TEST FILE =====
Hello from LAB2 JFFS2

===== PERSISTENCE TEST =====
Persistence test LAB2

After unmount + remount:
total 5
drwxr-xr-x 3 root    root      0 Jan 1 1970 .
drwxr-xr-x 3 root    root      0 Jan 1 1970 ..
-rw-r--r-- 1 root    root      0 Oct 8 02:31 .
-rw-r--r-- 1 nguyendangquang nguyendangquang 22 Sep 29 19:10 persistence.txt
-rw-r--r-- 1 nguyendangquang nguyendangquang 22 Sep 29 19:10 test.txt
Persistence test LAB2

===== PART C COMPLETE =====

```

Figure 6. JFFS2 erase, write, mount, read and persistence test.

3.4 Part D - BusyBox Init and Automatic Driver Loading

BusyBox init executes /etc/init.d/rcS during boot. The script mounts procfs, sysfs, tmpfs and devtmpfs, loads /lib/modules/5.15.0/lab2_driver.ko, and creates /dev/lab2. The boot log shows the driver registered with major 240, procfs was created, sysfs attributes were created, and boot continued normally.

```

nguyendangquang@nguyendangquang-Bellator:~$ grep -A 10 "Run /sbin/init as init process" -e /embedded_lab2/logs/boot_log.txt
Run /sbin/init as init process
Loading LAB2 driver...
lab2_driver: loading out-of-tree module taints kernel.
lab2: registered with major 248
lab2: module loaded successfully
lab2: /proc/lab2_info created
lab2: sysfs attributes created
LAB2 driver loaded
Boot completed: welcome to Embedded Linux Lab 2
Please press Enter to activate this console. Input: imxfs2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000/motherboard-bus@40000000/bus@40000000/motherboard-bus@40000000:1ofcp407,00000000/10007000.knl/ser1a1/input/input2

```

Figure 7. QEMU boot log showing automatic driver loading.

The test script contains the sequence used to load/check the module, create the device node, perform write/read tests, and inspect dmesg.

```

nguyendangquang@nguyendangquang-Bellator:~$ head -30 -e /embedded_lab2/logs/driver_test.txt
echo "=== LAB-02 Driver Test ==="

echo "[1] Load module"
insmod /lib/modules/5.15.0/lab2_driver.ko

echo "[2] Check module"
lsmod | grep lab2

echo "[3] Create device node"
mknod /dev/lab2 c 248 0
chmod 666 /dev/lab2

echo "[4] Write data"
echo "Hello from userspace, LAB-02!" > /dev/lab2

echo "[5] Read data"
cat /dev/lab2

echo "[6] Multiple write/read"
echo "Test #23" > /dev/lab2
cat /dev/lab2

echo "Test ABC" > /dev/lab2
cat /dev/lab2

echo "[7] Kernel log"
dmesg | grep lab2

echo "=== TEST DONE ==="
=== LAB-02 Driver Test ===

```

Figure 8. Driver test procedure.

The final output directory contains the rebuilt initramfs_lab2.cpio.gz and the four required evidence logs.

```

nguyendangquang@nguyendangquang-Bellator:~$ ls -lh -e /embedded_lab2/output/
total 28K
-rw-rw-r-- 1 nguyendangquang nguyendangquang 14K Oct 8 02:28 boot_log.txt
-rw-rw-r-- 1 nguyendangquang nguyendangquang 2.3K Oct 8 02:29 driver_test.txt
-rw-rw-r-- 1 nguyendangquang nguyendangquang 1.7K Oct 8 02:33 mtd_jffs2.txt
-rw-rw-r-- 1 nguyendangquang nguyendangquang 634 Oct 8 02:29 procfs_test.txt
-rw-rw-r-- 1 nguyendangquang nguyendangquang 1.1K Oct 8 02:24 /home/nguyendangquang/embedded_lab2/output/initramfs_lab2.cpio.gz

```

Figure 9. Final LAB-02 output and log files.

4. File System Comparison

File system	Writable	Wear leveling	Compression	Boot / mount	Suitable device
ext4	Yes	No built-in raw-flash wear leveling	Optional at other layers	Fast on block devices	eMMC, SD, HDD/SSD
JFFS2	Yes	Yes for raw flash	Yes	Slower as flash size grows because of scanning	Small raw NOR/NAND
cramfs	No	N/A	Yes	Fast, simple read-only image	Read-only firmware
SquashFS	No	N/A	High compression	Fast read-only root filesystem	Firmware/rootfs images
UBIFS	Yes	Works with UBI wear-leveling layer	Yes	Scales better than JFFS2 on larger NAND	Medium/large raw NAND

Recommendation for a 32 MiB raw NAND IoT device: JFFS2 is appropriate for a small writable raw-NAND filesystem and directly matches this laboratory. For an OTA-oriented production design, a robust layout can separate a compressed read-only firmware/rootfs image from writable persistent data and use an A/B update strategy when storage permits. Within the filesystems studied here, JFFS2 remains a reasonable choice for the small writable NAND partition.

5. Analysis Questions

5.1 Why `copy_to_user()/copy_from_user()` instead of `memcpy()`?

Kernel space and user space use different protected address spaces. A user pointer cannot be treated as an ordinary trusted kernel pointer. `copy_from_user()` safely copies data from a user buffer into kernel memory, while `copy_to_user()` safely copies kernel data back to user space and handles invalid or inaccessible user addresses according to kernel rules. Using `memcpy()` directly with an arbitrary user pointer can dereference an invalid address, bypass the required user-access checks, and can cause a kernel fault or security problem.

5.2 Why is a mutex required for `device_buffer`?

`device_buffer` is shared state. Without a mutex, two processes can execute write operations at the same time. For example, process A may begin copying 'AAAA...' while process B begins copying 'BBBB...' and updating `data_length`. The final buffer and length could represent different writes or partially mixed data. A mutex allows only one critical section at a time, so buffer contents, data length and related state remain consistent.

5.3 `procfs` versus `sysfs`

`procfs` was originally designed mainly for process and kernel-status information, while `sysfs` represents kernel objects, devices, classes and their attributes in a structured hierarchy. New device attributes are normally better represented in `sysfs` because each attribute has a clear object relationship and usually a simple one-value-per-file interface. In this driver, `/proc/lab2_info` is useful as a human-readable summary, while `buffer_len`, `open_count` and `last_data` naturally belong to the `lab2` device object under `/sys/class/lab2_class/lab2/`.

5.4 Why must NAND erase an erase block before rewriting?

NAND flash cannot freely overwrite bytes or sectors like a block device abstraction. Programming changes cells in one direction, and restoring them for reuse requires an erase operation over an entire erase block. Therefore the 16 KiB erase block in this experiment must be erased before it is reused. JFFS2 is log-structured: it writes new nodes sequentially rather than overwriting existing data in place, treats old nodes as obsolete, and later performs garbage collection to reclaim erase blocks. This design fits raw flash constraints and supports wear distribution.

5.5 What happens if `insmod` fails in `rcS`?

With the current `rcS`, boot continues because `insmod` is followed by an `if` statement that checks its exit status; failure prints an error message rather than terminating the `init` script. A fatal error is one that should stop the system or enter a recovery shell, while a non-fatal error is reported and boot is allowed to continue with reduced functionality. A stronger `rcS` can log the failure, avoid creating `/dev/lab2` when the module is unavailable, and explicitly choose whether to continue or enter a recovery shell depending on whether the driver is essential.

Example improved handling:

```

if insmod /lib/modules/5.15.0/lab2_driver.ko; then
    echo "[LAB-02] driver loaded OK"
    [ -c /dev/lab2 ] || mknod /dev/lab2 c 240 0
    chmod 666 /dev/lab2
else
    echo "[LAB-02] ERROR: driver load failed" >&2
fi

```

6. Comments, Difficulties and Solutions

Issue	Cause	Solution
test_procfs.sh was missing from the rootfs	The original initramfs did not contain the Part B test script.	Created test_procfs.sh, made it executable, rebuilt initramfs_lab2.cpio.gz and verified that the script was present.
mtd0 was not the NAND simulator	The host already exposed mtd0 as a read-only BIOS NOR device.	Loaded nandsim and verified /proc/mtd and mtdinfo. The simulator appeared as mtd1; all erase/write operations were performed only on /dev/mtd1.
Device node already existed during driver test	rcS automatically created /dev/lab2 before test_driver.sh was run.	Recognized the mknod warning as non-fatal; the driver was verified through successful write/read operations. A robust script can use '[-c /dev/lab2] mknod ...'.

7. Conclusion

LAB-02 was completed successfully on the ARMv7-A QEMU environment. The custom character driver loaded with major 240 and correctly handled user-space read/write operations. procfs and sysfs exposed live driver state, the JFFS2 experiment demonstrated erase/write/mount/persistence behavior on a 32 MiB simulated NAND device, and BusyBox rcS automatically loaded the driver during boot. The generated logs and rebuilt initramfs provide reproducible evidence for all four practical parts.